Introduction to Programming

Readings: Structures
Reading • 1h

Live Session: Structures
Reading • 1h

Structures
Discussion Prompt • 30 min

Dummy Lab for Live Session
Lab • 1h

Practice Lab Solutions
Reading • 10 min

Staff Graded Assignment -2
Programming Assignment • 1h

Live session
Reading • 10 min

Recording of Introduction to Programming: Week 7 - Live Session on 26-10-23 19:32:57 [51:12]
Video • 51 min

Module 9
Pointers

Programming assignment

Staff Graded Assignment -2

Launch lab



Instructions | My submissions | Discussions

What to expect

Due Nov 15, 11:59 PM IST

Unlimited attempts

Note:

The graded quizzes are auto-graded and count for 20% of your overall course grades. You will have one attempt for each graded quiz, allowing you to take the assessment only once. The minimum pass threshold for the quiz is solely for guidance. You can compare your quiz results against the minimum pass threshold to reflect on your learning proficiency on the topics covered in the quiz. It is important to note that per the institute's education policy, your quiz score will be used, along with the other evaluation components, to compute your final grade for the course using the relative grading method.

Assignment Overview

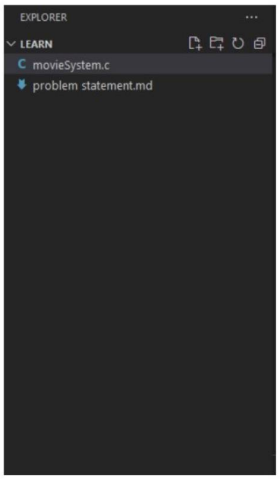
This is an auto-graded lab assignment based on the concepts of structures in C. You must attempt all questions in this assignment through Coursera Labs that test your understanding of the coding and programming concepts. Coursera Labs provides access to the lab environment to execute the related programming code.

Time Estimate

- You have **one hour** to complete and submit the auto-graded lab assignment.
- You can submit your lab assignment only **once**.

How to Attempt the Auto-Graded Assignment

- Click the **Auto-Graded Lab Assignment** button to access the Coursera Labs environment to run and execute the required code.
- The auto-graded assignment questions have been provided in the assignment brief below and preloaded in Coursera Labs.
- You can access the questions files from the left panel in the lab. Please refer to the image below



Assignment Questions:

AutogradedLab_Question1

Planetary Colonisation Evaluation Module

Write a program to implement a planetary colonization evaluation module using a structure array, for space scientists who are evaluating the suitability of other planets for human colonization.

This program should be capable of handling the following operations:

The program should prompt the user to enter the number of planets. This will determine the size of the array of structures.

Each structure should have the following fields:

- **planetName** (array of characters) to store the name of the planet,
- **distanceFromEarth** (float) for the distance of the planet from Earth, in light-years,
- **gravity** (float) for the planet's gravity, as a factor of Earth's gravity 'g' (1.2 implies 1.2g, 0.5 implies 0.5g, etc)
- **atmosphere** (a nested structure) that includes oxygen level, carbon dioxide level, and nitrogen level in percentages (each stored as a float).
- **habitable** (character) which is automatically **evaluated by the program**; a planet is habitable if its gravity is between 0.8 and 1.2 g (both inclusive) and its oxygen level is between 18% and 25% (both inclusive). This field should store whether the planet is "Y" (Yes) or "N" (No) for habitability, thus.

The program should present a menu to the user with the following options:

a) Find and print all **habitable** planets, sorted by their distance from Earth in ascending order.

b) Search for and print the names of all planets where the oxygen level is **above** a user-given threshold and the carbon dioxide level is **below** another user-given threshold.

c) Search for planets by a **keyword** or substring in their name (the keyword will be provided by the user) and print the planet names that match.

d) Exit the program.

The program should continue to display the menu after each operation until the user chooses to exit.

Important considerations for the operations:

- Assume that the planet name consists of just one word, and there are no spaces in the name.
- The menu and other formatting has already been provided to you in the template code. Only write in the space provided to you (denoted by the instructions in comments).
- When sorting planets by distance, use the Selection Sort algorithm.
- When searching by keyword or substring in planet names, use the **strstr** function to match the keyword.
- If the user enters an invalid option, the program should display an "Invalid input" message.

Please go through the test case given below to design your MENU and understand how the expected outputs should be printed.

Testcase

```
Enter the number of planets: 3
Enter Planet Name: Mercury
Enter Distance from Earth (in light-years): 4.3
Enter Planet Gravity (in terms of g): 1.15
Enter Atmosphere Composition (Oxygen, Carbon Dioxide, Nitrogen) as percentages: 19
0.03
75
Enter Planet Name: Earth
Enter Distance from Earth (in light-years): 0
Enter Planet Gravity (in terms of g): 1
Enter Atmosphere Composition (Oxygen, Carbon Dioxide, Nitrogen) as percentages: 21
0.02
78
Enter Planet Name: Neptune
Enter Distance from Earth (in light-years): 12
Enter Planet Gravity (in terms of g): 0.7
Enter Atmosphere Composition (Oxygen, Carbon Dioxide, Nitrogen) as percentages: 16
0.05
80

MENU
1. Find and print all habitable planets
2. Search by oxygen and carbon dioxide levels
3. Search by keyword in planet names
4. Exit
1
Habitable planets (sorted by distance from Earth):
Earth - 0.00 light-years
Mercury - 4.30 light-years

MENU
1. Find and print all habitable planets
2. Search by oxygen and carbon dioxide levels
3. Search by keyword in planet names
4. Exit
3
Enter a keyword to search in planet names: Ear
Planets matching the keyword "Ear":
Earth

MENU
1. Find and print all habitable planets
2. Search by oxygen and carbon dioxide levels
3. Search by keyword in planet names
4. Exit
4
Exiting...
```

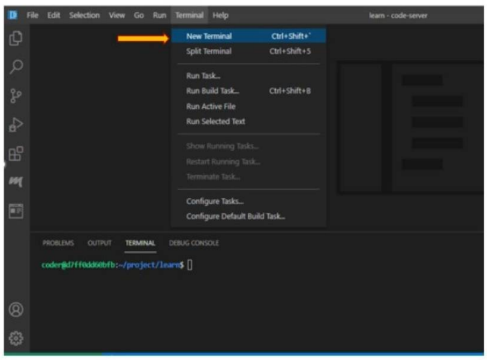
[IMPORTANT] Notes about Input/Output Format:

Please be very careful and strictly adhere to the input/output format specified.

- Do not modify any of the pre-existing printf() statements.
- For each of the cases (1 to 3), print the output exactly as specified. Go through the test case for more clarity.

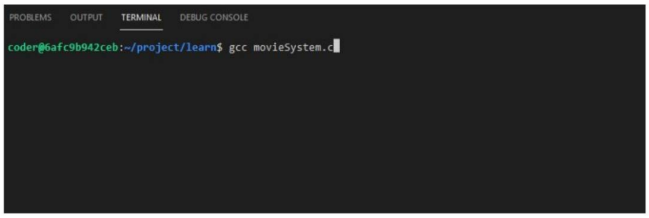
Compilation/Execution Steps:

- Open a new terminal by clicking on the menu bar and selecting ->**Terminal ->New Terminal**



Compile the C file using the following command in the terminal:

gcc <Filename.c> (Example: gcc planets.c)



Run the C file using the following command in the terminal once the a.out file is created:

./a.out

Every time you make a change in your program or the filename.c file, it is required to compile the program again using "gcc filename.c" to reflect that change in your a.out. Otherwise, you will be accessing an a.out file of the previous version of your code, and the changes you made will not be reflected in the output.

Reference Video for Compilation/Execution Steps:

https://www.coursera.org/learn/introduction-to-programming/lecture/LOX9s/executing-a-c-program-using-coursera-labs

Total score and distribution of points:

- Total points—30 points

Report an issue